NOTES · PART 2

THE LOOP AND THE KNOWLEDGE STEP

Memorandum on Agentic AI Applications · 24 Oct 2026 · continues Part 1

Contents

IX. What Part 2 adds

X. Inside the loop

XI. Inside the knowledge step (RAG and context)

Part 1 showed what an agentic app is made of and pointed each part to a paper. From here on, the notes open those papers and show how each part works inside: the steps, a diagram of the flow, and the few numbers worth knowing. Red underlined titles name the paper or guide; the addresses are in Part 1, section VIII, and the updates in Part 5, section XIX.

IX. WHAT PART 2 ADDS

Part 1 and the 30 paper memos cover the same reading list from two sides. This part joins them.

PART 1 GAVE YOU	PART 2 ADDS	SECTION
The one loop (think, tool, result)	The exact message format and how tools are chosen	X
Chat with your documents	The RAG pipeline, self-checking RAG, and the new topic: context engineering	XI
Fill a fixed form, stream the answer	Why the JSON can't break, and the order of stream events	XII
Five rules, human approval, long jobs	Guardrail layers, the 12 factors, how pause/resume and crash-recovery work	XIII
Router, research team, crew	Handoff vs manager, the real costs, how crews pass documents	XIV
Coding / file agent	How coding agents are tested and the commands they use	XV
Voice agent, local agent	Inside Whisper, the two voice designs, how local models fit	XVI
OWASP and prompt injection links	How the attack works, and the full risk list mapped to your app	XVII

Shapes used in the diagrams: a shaded box is the AI model or agent, a red dashed box is a risk or a guard, plain boxes are people, tools, data and results. A dashed arrow means "go back and repeat".

X. INSIDE THE LOOP

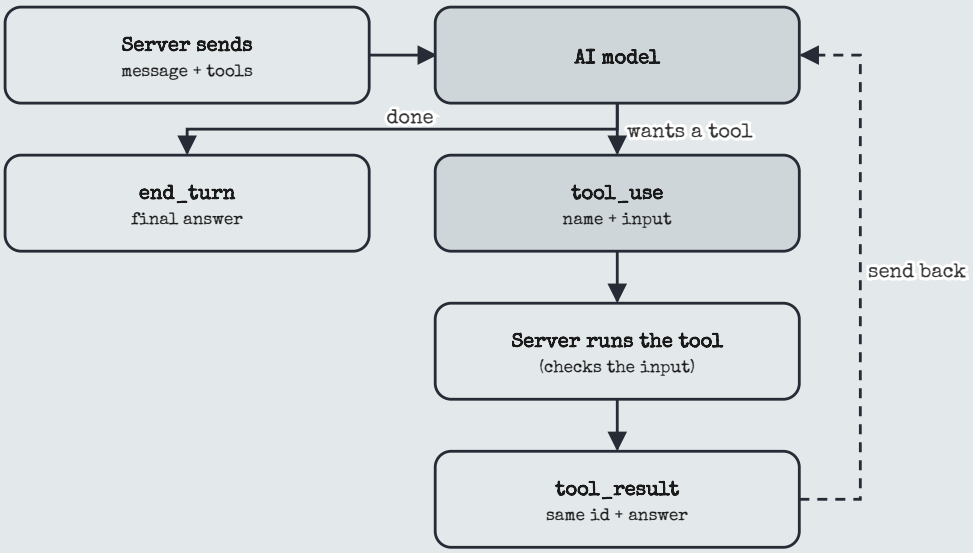
1. The loop as the ReAct paper wrote it

The model writes three kinds of lines, again and again:

```
Thought: I need the order data first.
Action: lookup_order[123]
Observation: Shipped, arrives 22 Oct.
Thought: Now I can answer.
Action: finish[Your order arrives on 22 October.]
```

- Your code stops the model after each *Action*, runs it, and pastes the *Observation* back in.
- Mixing thinking with real results makes up far fewer facts than thinking alone. On two decision tasks (ALFWorld and WebShop) this beat older trained methods by 34 and 10 points.

2. The same loop in real code (tool use)



The server's job in every agent app: send tools, run the tool the model asks for, send the result back.

1. Each tool has a name, a description (3 to 4 clear sentences) and an input shape (JSON Schema).
2. The model answers with a `tool_use` block. It can ask for several tools at once.
3. The server runs the tool and replies with `tool_result` carrying the same id. If it failed, mark it as an error so the model can fix itself.
4. Repeat until the model stops with `end_turn`.

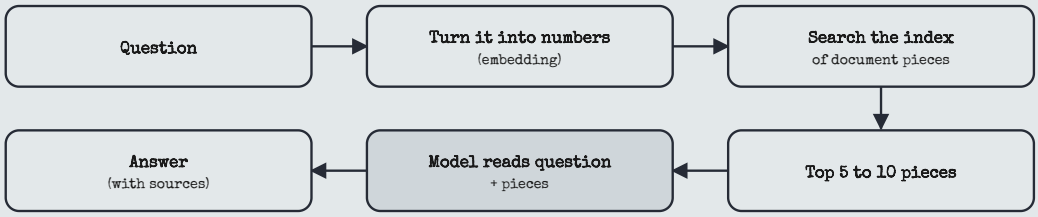
3. How the model picks a tool

- MRKL: a router sends each part of the job to an expert (calculator, database, another model). The hard part is pulling the right inputs out of the user's words, so let code do the exact work.
- Toolformer: the model learned tool calls by itself. It kept a practice call only if the result made the following text easier to predict. A 6.7B model with tools beat the 175B GPT-3 on maths and fact questions.
- Good tools matter more than clever prompts: few tools that don't overlap, clear names (`user_id`, not `user`), short answers, error messages that say how to fix the problem. Test tools with a list of real tasks, then improve them.

> READ THE PAPER
>
> ▸ ReAct (Yao et al., 2022): the Thought, Action, Observation loop
>
> ▸ Tool use with Claude (Anthropic docs): tool_use and tool_result in code
>
> ▸ Writing effective tools for agents (Anthropic): design tools, then test them with evals

XI. INSIDE THE KNOWLEDGE STEP (RAG AND CONTEXT)

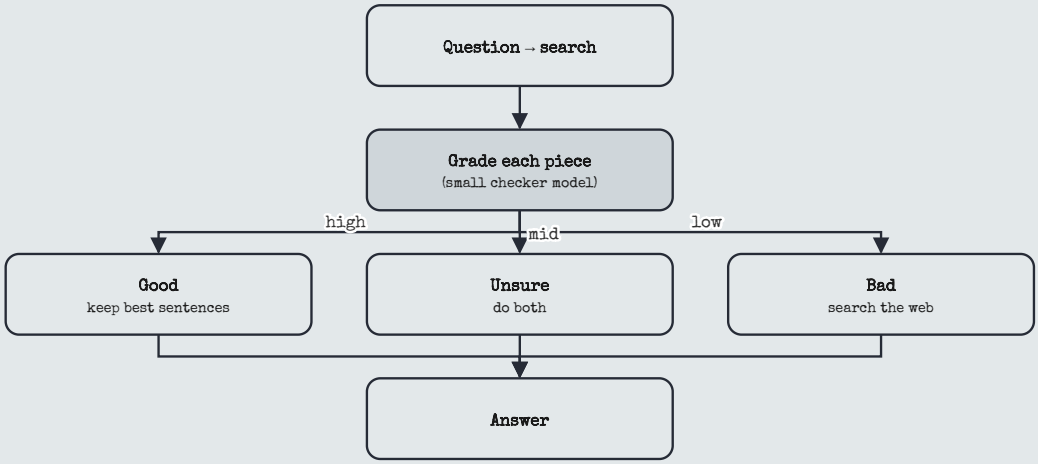
1. The original RAG pipeline



Retrieval-augmented generation: find pieces first, then answer from them.

- The original paper split Wikipedia into 21 million pieces of 100 words, searched them, and let a small model answer from the top 5 to 10.
- Swapping the index for a newer one updated the model's knowledge without retraining. This is why your data stays fresh.

2. Careful RAG: two ways to check the search



Corrective RAG: grade the search results, then choose what to do.

- Self-RAG: the model writes small check-marks into its own answer: *Search?* (yes/no), *Relevant?*, *Supported?*, *Useful?* (1 to 5). It searches only when needed and keeps the best-supported answer.
- Corrective RAG: a small grader scores the results. Good means trim them to the useful sentences. Bad means throw them away and search the web. Unsure means do both.
- You can copy both with prompts: ask "do I need to search?" first and "is this answer backed by the sources?" last.

3. New topic: context engineering

Part 1 didn't cover this. The context window is the agent's working memory, and it has a budget. The more you put in, the worse the model gets at using it. So choose, at every turn, the smallest set of useful words.

PROBLEM	WHAT TO DO	EXAMPLE
Instructions too strict or too vague	Write them at the "right height": clear sections, only rules that fix real mistakes	System prompt with headings
Too much data loaded up front	Load it when needed: keep file names or links, fetch with a tool	Search files, then open one
Long chat fills the window	Compaction: summarise, keep decisions and open problems, drop old tool results	Long coding session
Agent forgets across sessions	Notes outside the window that it reads back	NOTES.md, to-do list
One job needs lots of reading	Sub-agents read in their own window and return a short summary	Research helpers

> READ THE PAPER
>
> - Retrieval-Augmented Generation (Lewis et al., 2020): the original pipeline
> - Self-RAG (Asai et al., 2023): search only when needed, check support
> - Corrective RAG (Yan et al., 2024): grade results, search the web if bad
> - Effective context engineering for AI agents (Anthropic): the new topic in this part

* * *

Continued in Part 3.